

Serialization Vulnerabilities

Serialization Vulnerabilities - Author not stated, rce-serialization-dotnet.

- Published: date not stated
- Original: <https://johniwasz.github.io/rce-serialization-dotnet/serialization/serialization.html>
- Preserved from: <https://johniwasz.github.io/rce-serialization-dotnet/serialization/serialization.html> (preserved-copy) on 2026-08-04
- Licence: unknown

Rights remain with the original author and publisher. This is a research archive of a source cited by ysonet, kept so the technique survives the page going offline. To read the original, follow the link above.

Content

UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

Serialization Vulnerabilities | rce-serialization-dotnet

rce-serialization-dotnet

Serialization Vulnerabilities

Serialization vulnerabilities are not just limited to the `BinaryFormatter`. Any message that includes the type to deserialize poses a threat irrespective of method of serialization. `Newtonsoft.Json` and `MessagePack[1]()` are also susceptible.

Deserializing messages sent from an untrusted source introduces risk. Compromised messages processed by unsafe deserializers could result in an error at best or a remote code execute exploit at worst. OWASP defines this as:

Data which is untrusted cannot be trusted to be well formed. Malformed data or unexpected data could be used to abuse application logic, deny service, or execute arbitrary code, when deserialized[2]().

This project demonstrates serialization vulnerabilities using `Json.NET` and the `BinaryFormatter`.

The `Todo` project is based on the starter tutorial available here:

[Create a controller based API](#)

These examples were assembled based on serialization vulnerabilities highlighted in the [ysoserial.net](#) git repo.

Deserialization and Gadget Chains

Most serialization exploits utilize two components.

- Type Specification. Messages that define the type to deserialize allow the caller use a vulnerable type.
- Gadget. An exploitable type that includes a capability to launch a command specified in the message.

Exploitable types are referred to as **gadgets**. These allow malicious code to be launched using a constructor, property, method, or a code snippet. **Gadgets** can be a native class or injected into the application. These may not be obvious if a class has a deeply nested hierarchy. In C#, classes that include `object` types or generics can be suspectable. For example, many public APIs include a capability to store metadata or arbitrary data that's captured with a `Dictionary<string, object>`. This opens a door for potential exploitation.

Prerequisites

Install [Visual Studio 2022](#).

Install .NET Framework 4.8.1:

```
winget install Microsoft.DotNet.Framework.DeveloperPack_4 -v 4.8.1
```

Install .NET 8:

```
winget install Microsoft.DotNet.SDK.8
```

Install Git:

```
winget install Git.Git
```

Ysoserial.net

[Ysoserial.net](#) creates malicious serialized payloads for a variety of libraries including [Json.NET](#), `MessagePack`, and `BinaryFormatter`.

The payloads cannot be programmatically generated using these serializers with native implementations. Ysoserial.net manipulates the serialized message to inject a malicious payload. Deserialization results in a runtime exception; however, a remote code execution exploit is triggered.

-

Enable .NET 3.5 on Windows using the instructions here: [Enable .NET 3.5 on Windows](#). This is required to build ysoserial.net. It exploits vulnerabilities in prior versions of the .NET Framework.

-

Open a command prompt and navigate to:

```
%userprofile%\source\repos
```

-

Clone the ysoserial.net repo.

```
git clone https://github.com/pwntester/ysoserial.net.git
```

-

Open the solution in Visual Studio and build.

-

Navigate to the debug directory in a command prompt.

```
%userprofile%\source\repos\ysoserial.net\ysoserial\bin\Debug
```

-

Execute `ysoserial --help`. The executable may not be available. It is considered malware by most malware scanners. Check Windows Security or your active malware scanner for detected threats and remove `ysoserial.exe` from quarantine.

| a. If `ysoserial.exe` is not available, navigate to Windows Security and select Virus & Threat Protection. Review threat history and find `ysoserial.exe`. Select Action | Restore. | |

Protection history

View the latest protection actions and recommendations from Windows Security.

Filtered by: Quarantined Items

Filters 



Threat quarantined

4/7/2024 10:59 AM

Severe 

Detected: VirTool:MSIL/Vusrlize.A!MTB

Status: Quarantined

Quarantined files are in a restricted area where they can't harm your device. They will be removed automatically.

Date: 4/7/2024 10:59 AM

Details: This program is used to create viruses, worms or other malware.

Affected items:

file: C:\Users\ [redacted] \repos\ysoserial.net\ysoserial\bin\Debug\ysoserial.exe

[Learn more](#)

Actions 

Restore 

Remove

b. Alternatively, ysoserial can be excluded from threat protection using these instructions: [Add an exclusion to Windows Security](#).

-

Optionally, add the Debug directory to your %PATH%.

Take a moment to review the [Usages](#) of `ysoserial` parameters before embarking into the projects.

Clone rce-serialization-dotnet Repo

The [rce-serialization-dotnet](#) repo includes the serializations projects and this documentation.

-

Open a command prompt and navigate to:

```
%userprofile%\source\repos
```

-

Clone the rce-serialization-dotnet repo.

```
git clone https://github.com/johniwasz/rce-serialization-dotnet.git
```

The projects are in the `src` subdirectory.

Serialization Vulnerability Repros

Each project reproduces a serialization vulnerability.

| Project | Description | | | [01-BinaryFormatterVulnerabilities](#) | BinaryFormatter vulnerability in .NET Framework 4.8 | | | [02-Framework-JsonVulnerabilities](#) | Json.NET vulnerability in .NET Framework 4.8 | | | [03-.NET-JsonVulnerabilities](#) | Json.NET vulnerability in .NET 8 | |

Load the solution in each project in Visual Studio, run the solution, and use the `requests.http` in the Solution Items folder to walk through the vulnerabilities.

-

MessagePack includes [Typeless deserialization](#) which embeds the type to deserialize in the message. `[↵]()`

-

[Deserialization of Untrusted Data](#)`[↵]()`

Archived from <https://johniwasz.github.io/rce-serialization-dotnet/serialization/serialization.html>. Rendered offline from the local Markdown copy.